

Architettura degli Elaboratori

Cache e memoria virtuale



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits: Alessandro
Checco, Andrea Sterbini, Iacopo Masi,
Claudio di Ciccio

[S&PdC]

5.6–5.8



Esercizi Esonero



SAPIENZA
UNIVERSITÀ DI ROMA

Esercizio A

Esercizio

Si realizzi in assembly RISC-V il seguente programma. Sia data in input da utente una stringa di caratteri S . La stringa è lunga al più 24 caratteri (non è necessario verificare questo vincolo) e consiste dei seguenti simboli: '0' (codice ASCII 0x30), '1' (codice ASCII 0x31), '/' (codice ASCII 0x2F). La sequenza termina quando viene letto il carattere '\n' (codice ASCII 0xA). Il programma deve considerare ogni sequenza di caratteri separata da '/' come una sequenza a sé, e stampare su una nuova linea la rappresentazione decimale dei gruppi di numeri binari presi tre a tre in base 8 (ottale) di tale sequenza. Qualora vi fossero in coda gruppi di caratteri che non formano un gruppo di tre, essi vanno ignorati. Se la sequenza S è completamente vuota, non dovrà essere scritto a schermo alcunché. Se, ad esempio, S è 101/111 allora dovrà essere scritto a schermo su due linee separate:

```
5
7
```

perché la prima terzina corrisponde al numero 5, cioè $(1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0)$ e la seconda terzina corrisponde al numero 7, cioè $(1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0)$.

Se, invece, S fosse 111/101000110/1111110 allora dovrà essere scritto a schermo su tre linee separate:

```
7
506
77
```

perché il primo gruppo di caratteri forma una terzina corrispondente a 7, il secondo ha tre terzine corrispondenti a 506, e il terzo ha due terzine corrispondenti a 77 e un carattere finale 0 che verrà ignorato visto che non contribuisce a una terzina.

Esercizio A

```
24 ##### Inizializzazione
25 #####
26 li s0, 47          # separatore "/"
27 li s1, 10          # newline
28 mv s2, a0          # indirizzo stringa
29
30
31 gruppoTre:
32     li t0, 3        # di quanto shiftare
33     li t2, 0        # accumulatore
34 loopGruppo:
35     addi t0, t0, -1  # contatore gruppo decrementa
36     lb t1, 0(s2)    # carica carattere
37     addi s2, s2, 1   # prossimo
38     beq t1, s1, exit # esci se \n
39     beq t1, s0, fineGruppo # se separatore, stampa a capo e ricomincia
40     addi t1, t1, -48 # trasformalo in intero
41     sll t1, t1, t0   # spostalo
42     add t2, t2, t1   # accumula
43     beq t0, zero, fineGTre # raggiunti tre, stampa numero e ricomincia
44     j loopGruppo
```

Inizializzare i parametri

Inizializzare blocco e accumulatore

Esercizio A

```
45  fineGTre:
46      mv a0, t2          # Carica il risultato per la stampa
47      li a7, 1           # Imposta stampa intero
48      ecall
49      j  gruppoTre
50  fineGruppo:
51      li a0, 10           # newline
52      li a7, 11          # Imposta stampa carattere
53      ecall              # Stampa newline
54      j  gruppoTre
55  exit:
56      li a7, 10          # Imposta uscita
57      ecall
```

Esercizio B

Esercizio

Si realizzi in assembly RISC-V il seguente programma. Sia data in input da utente una stringa di caratteri *S*. La stringa è lunga al più 24 caratteri (non è necessario verificare questo vincolo) e consiste dei seguenti simboli: '0' (codice ASCII 0x30), '1' (codice ASCII 0x31), '+' (codice ASCII 0x2B). La sequenza termina quando viene letto il carattere '\n' (codice ASCII 0xA). Il programma deve considerare ogni sequenza di caratteri separata da '+' come un numero binario ed effettuare l'operazione corrispondente di somma (+) come rappresentato nella stringa di input e stampare a schermo la rappresentazione decimale del risultato. I caratteri sono separati sempre da un solo separatore (+). Qualora la sequenza non contenesse alcun carattere, va considerata come il numero 0.

Se, ad esempio, *S* è 11+0000+1+0010 allora dovrà essere scritto a schermo:

6

perché la sequenza corrisponde in decimale all'espressione $3+0+1+2=6$.

Nella sezione dedicata al **text segment**, il programma deve avere nel comparto *main* il caricamento dei dati da input (già fornito nel file `program01.asm`: l'indirizzo base della stringa *S* verrà caricato nel registro `a0`), e il codice necessario a calcolare i valori e visualizzare i risultati sullo schermo.

Esercizio B

```
24 ##### Inizializzazione
25 #####
26 li s0, 0x2B # separatore "+"
27 li s2, 10 # newline
28 mv s3, a0 # indirizzo stringa
```

Inizializzare
i parametri

```
29
30 li t1, 0
31 li t2, 0
```

inizializza addendo
inizializza risultato

Inizializzare blocco
e accumulatore

```
32
33 scorriStringa:
34     lb t3, 0(s3)
35     addi s3, s3, 1
36     beq t3, s2, end
37     beq t3, s0, Somma
38     addi t3, t3, -48
39     slli t1, t1, 1
40     add t1, t1, t3
41     j scorriStringa
42
43 Somma:
44     add t2, t2, t1
45     li t1, 0
46     j scorriStringa
47
48 end:
49     add t2, t2, t1
```

Esercizio B

```
51 Print:
52     mv a0, t2      # Carica il risultato per la stampa
53     li a7, 1       # Imposta stampa intero
54     ecall
55
56     li a0, 10      # newline
57     li a7, 11      # Imposta stampa carattere
58     ecall          # Stampa newline
59
60     li a7, 10      # Imposta uscita
61     ecall
```


Sinossi

Problema:

- In un sistema multi-processo la gestione della memoria è ardua
- La memoria principale può non essere sufficiente

Requisito:

- Rendere lo spazio di memoria per i processi indipendente dal limite della memoria disponibile

IDEA:

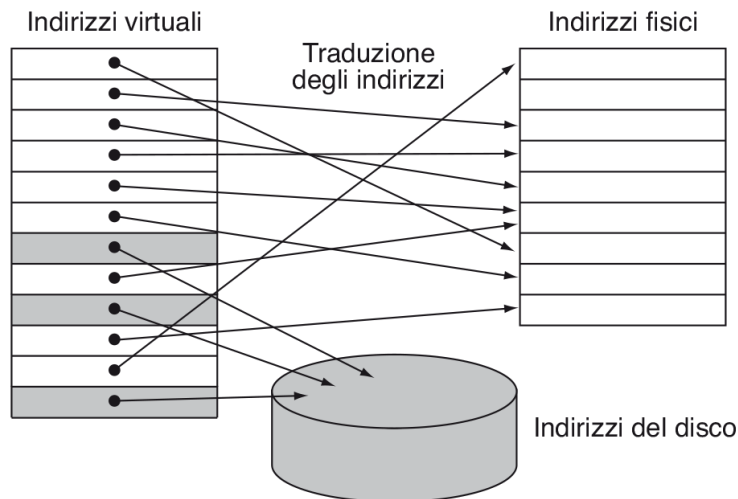
- 1 Gli **indirizzi** (istruzioni e dati) prodotti dalla CPU sono «**virtuali**»
 - vengono trasformati in **indirizzi «fisici»** per accedere alla vera posizione dei dati
- 2 La memoria è divisa in «pagine»
 - presenti in memoria fisica solo se necessario
(altrimenti possono essere memorizzate in una memoria più lenta)

Vantaggi:

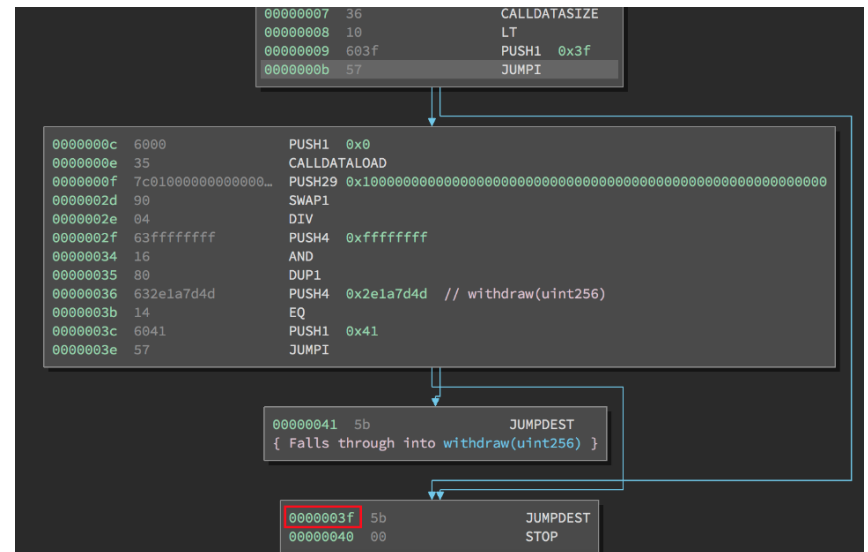
- Lo spazio di indirizzamento di un processo si semplifica
- Gli spazi di memoria dei processi vengono separati e protetti da accessi esterni

Differenza fra memoria virtuale e macchina virtuale

Memoria virtuale



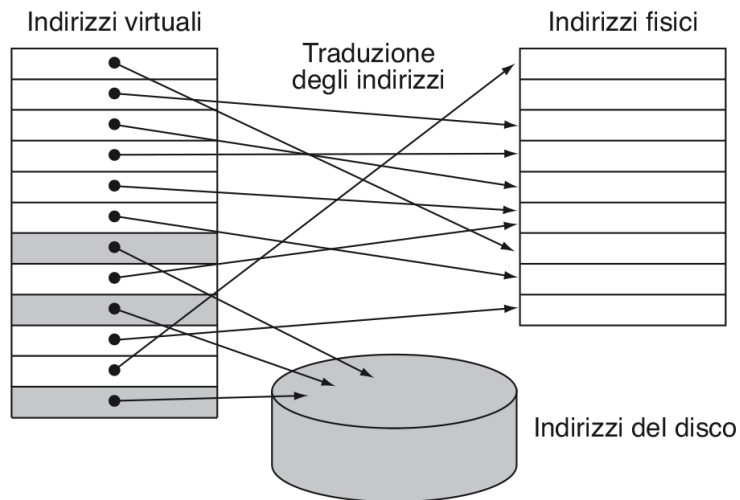
Macchina virtuale



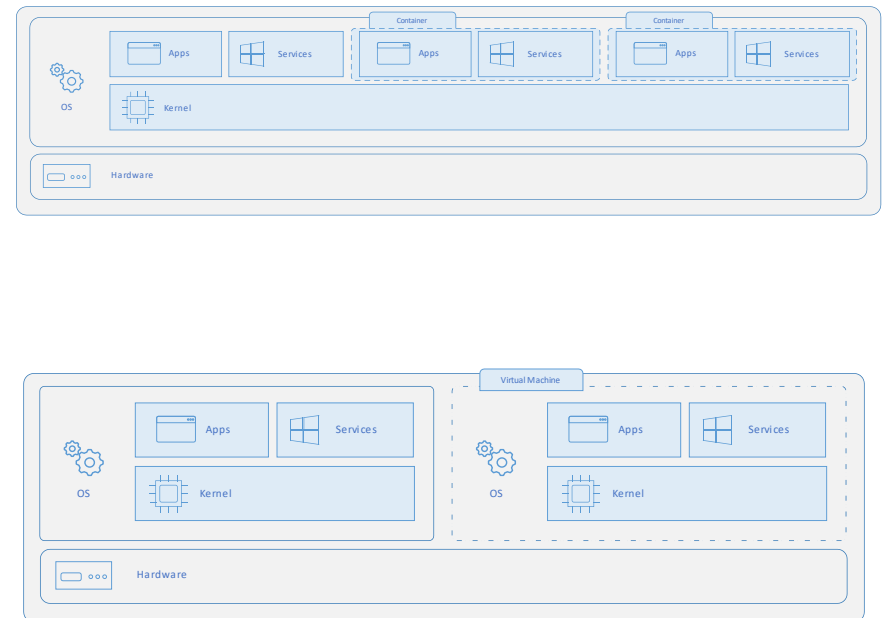
Tecnologia ideata negli anni 60,
abbandonata con i primi PC
e poi ripresa

Differenza fra memoria virtuale e macchina virtuale

Memoria virtuale



Macchina virtuale e container



Mapping degli indirizzi virtuali

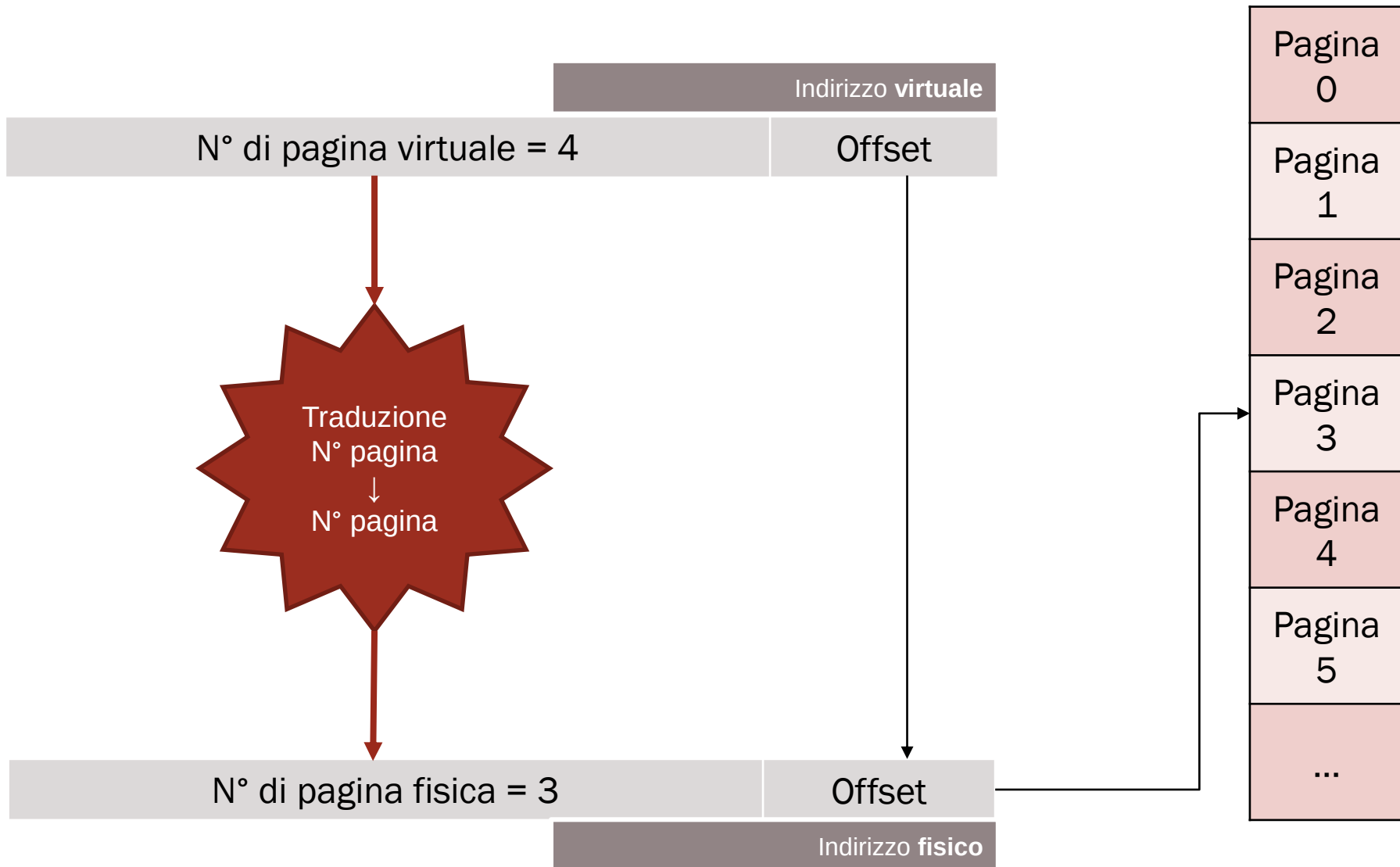


Tabella delle pagine (page table) del processo

Indirizzo della page table

In un registro

Indirizzo virtuale

N° di pagina virtuale = 4

Offset

Per ogni processo

Valid	Dirty	Used	N° di pagina fisica
0	0	0	Ind. in memoria di massa
1	1	0	3

Memoria di Massa

N° di pagina fisica = 3

Offset

Indirizzo fisico

Memoria principale

Pagina 0

Pagina 1

Pagina 2

Pagina 3

Pagina 4

Pagina 5

...

La lunghezza in bit del N° di pagina fisica può essere diversa da quella del N° di pagina virtuale

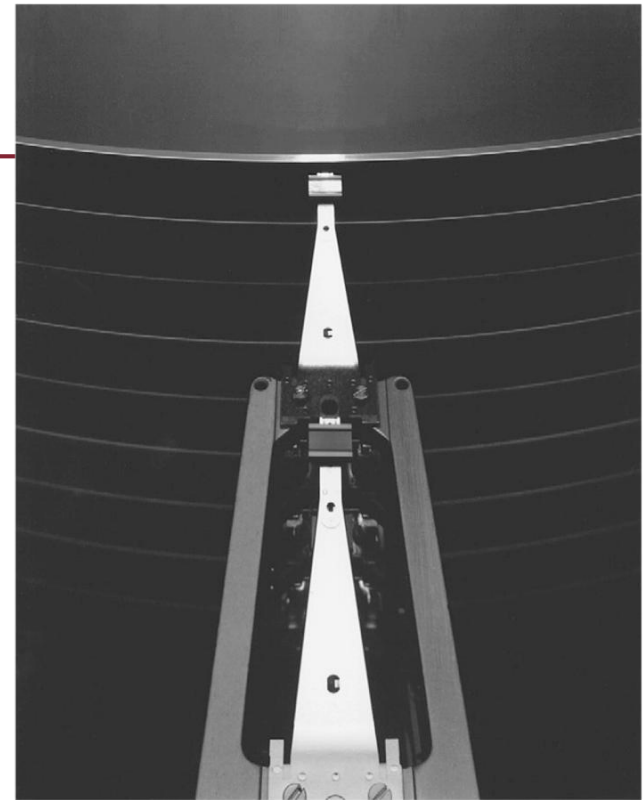
Accesso a un dato

Pagina già in memoria principale:

- **Valid = 1** → la tabella indica l'indirizzo in memoria principale
 - Sono necessari 2 accessi per ottenere il dato
 - 1 alla page table
 - 1 alla memoria fisica
 - Nota: 2 accessi si hanno nel **caso migliore**
→ Vanno resi più veloci!

Pagina non in memoria principale:

- **Valid = 0** → la tabella indica l'indirizzo in memoria di massa
- Viene lanciata un'eccezione di «**Page Fault**»
 - Richiede al sistema operativo che la pagina sia recuperata e inserita in memoria fisica
- Sono necessari **milioni di cicli di clock** per trovare e copiare il blocco dal disco



Politiche di rimpiazzo

Come per la cache occorre

1. **sostituire pagine già usate**
2. **mantenere la coerenza dei dati**

Politiche di rimpiazzo:

- **LRU (Least Recently Used)**
 - sostituire la pagina **utilizzata meno di recente**
 - Altro bit della Page Table:
 - **Used = 1** → la pagina è stata utilizzata di recente
 - approssima la politica di rimpiazzo LRU
- **LFU (Least Frequently Used)**
 - sostituire la pagina **utilizzata meno di frequente**
- **Random:**
 - sostituire una pagina a caso

Si ha tempo a disposizione → politiche più sofisticate sono possibili

- Piccoli miglioramenti comportano guadagni di tempo elevati

Politiche di scrittura

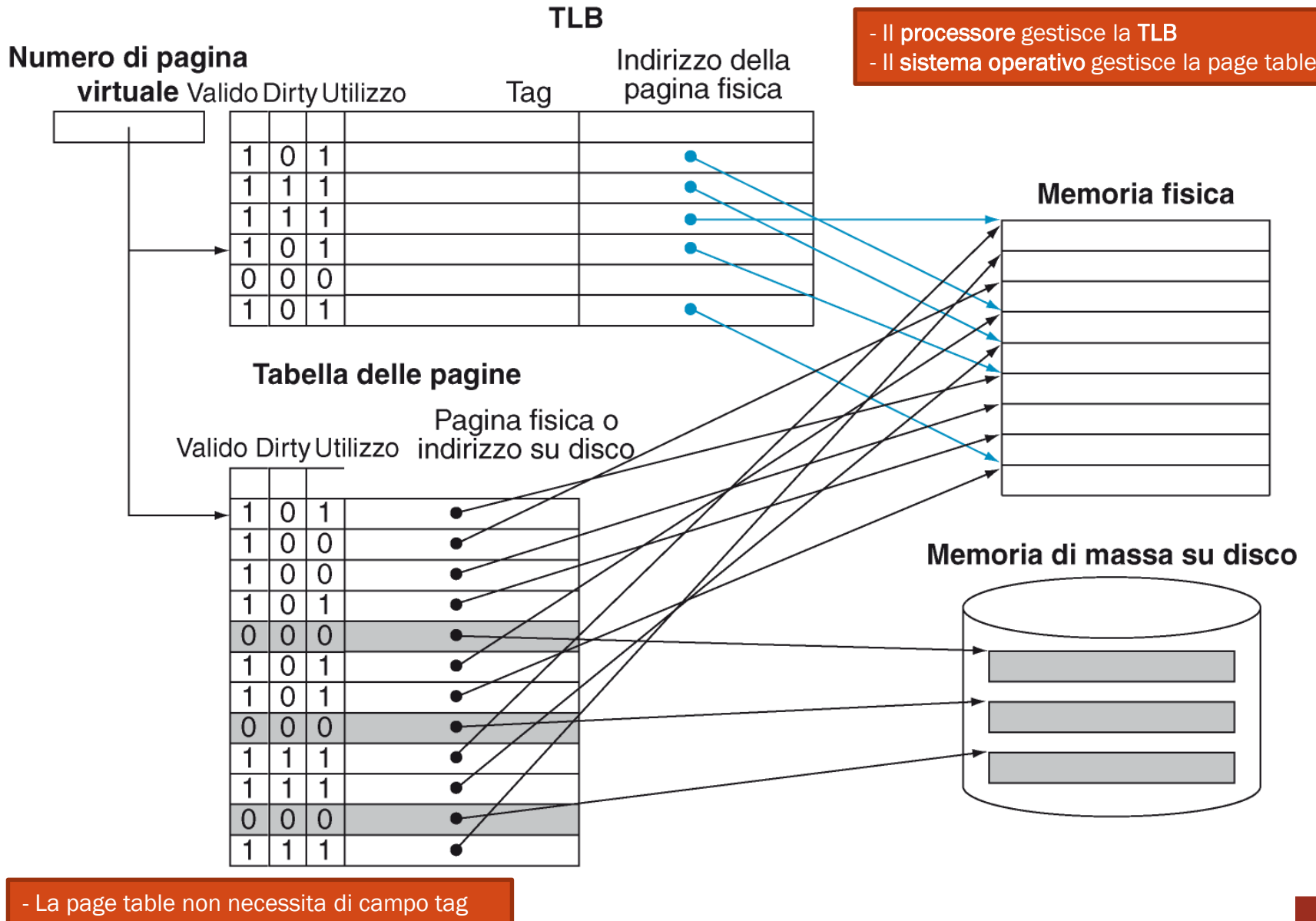
Come per la cache dobbiamo

1. sostituire pagine già usate e
2. **mantenere la coerenza dei dati**

Politica di scrittura:

- **Write Back**
 - Tempi sono troppo lunghi per il Write Through
 - Persino per la cache: aveva bisogno di **buffer**
 - Altro bit della Page Table:
 - **Dirty = 1** → la pagina è stata modificata in memoria
 - necessario salvarla in memoria di massa
prima di eliminarla dalla memoria principale

Accesso più veloce con il Translation Lookaside Buffer (TLB)



TLB + Cache del dato (processore FastMATH)

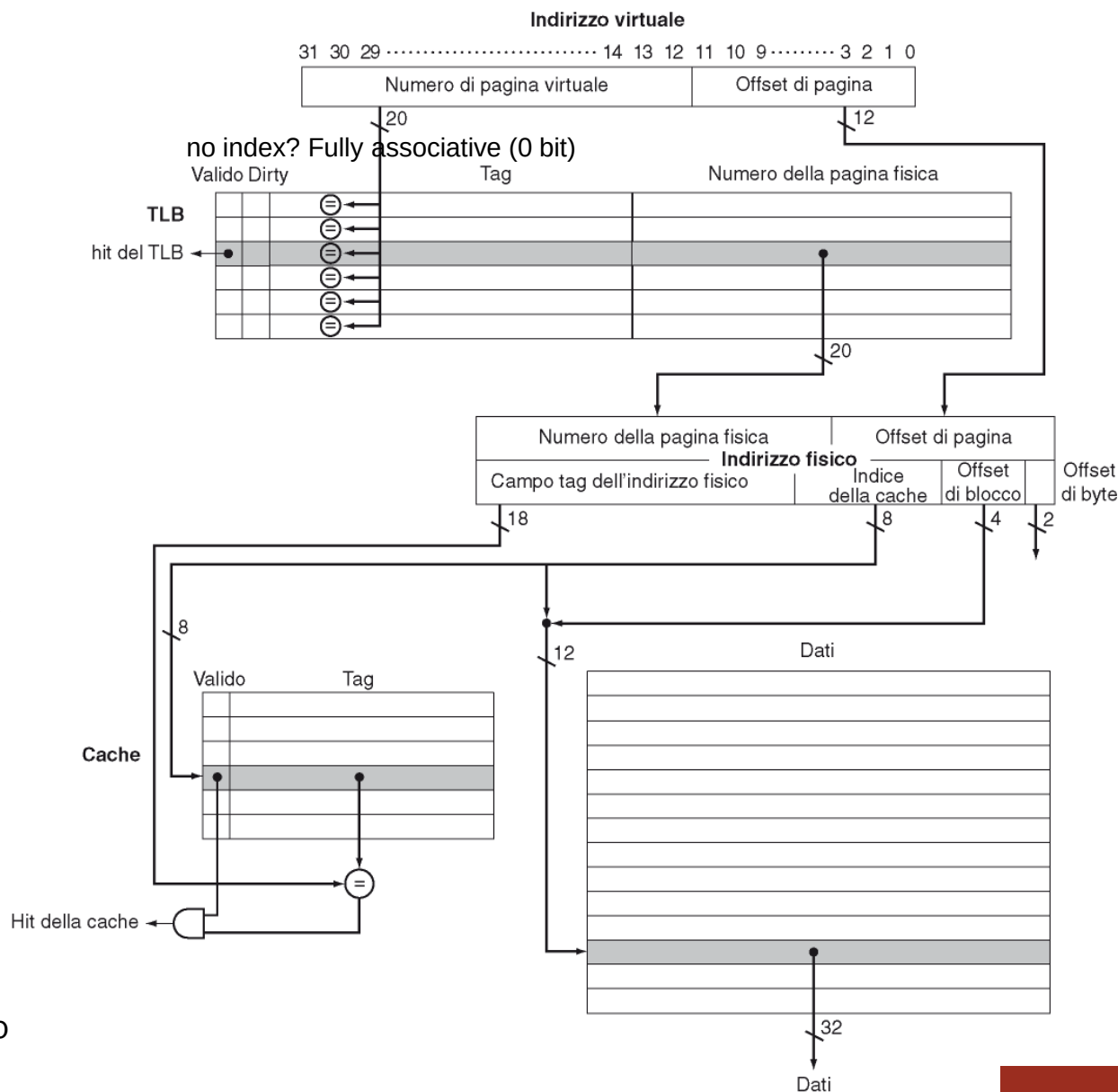
Per rendere ancora più veloce

l'accesso nel caso migliore
si opera inserendo una
cache sui due accessi

- **TLB:** cache dell'accesso alla Page Table
- **Cache:** dell'accesso al dato (in memoria fisica)

In genere la cache sul dato può essere indicizzata:

- dall'indirizzo virtuale
 - Va svuotata durante il context switch
- oppure dall'indirizzo fisico (come in FastMATH)
 - Va svuotata al caricamento di una nuova pagina



Visione d'insieme

In genere la cache sul dato può essere indicizzata:

- dall'indirizzo fisico
 - Va svuotata al caricamento di una nuova pagina
- oppure dall'indirizzo virtuale
 - Va svuotata durante il **context switch**

Possibili approcci:

CPU $\xrightarrow{\text{virtuale}}$ Page Table $\xrightarrow{\text{fisico}}$ RAM

CPU $\xrightarrow{\text{virtuale}}$ Cache [$\rightarrow$ TLB [$\rightarrow$ Page Table] $\xrightarrow{\text{fisico}}$ RAM]]

CPU $\xrightarrow{\text{virtuale}}$ TLB [$\rightarrow$ Page Table] $\xrightarrow{\text{fisico}}$ Cache [$\rightarrow$ RAM]

(Caso del processore FastMATH)

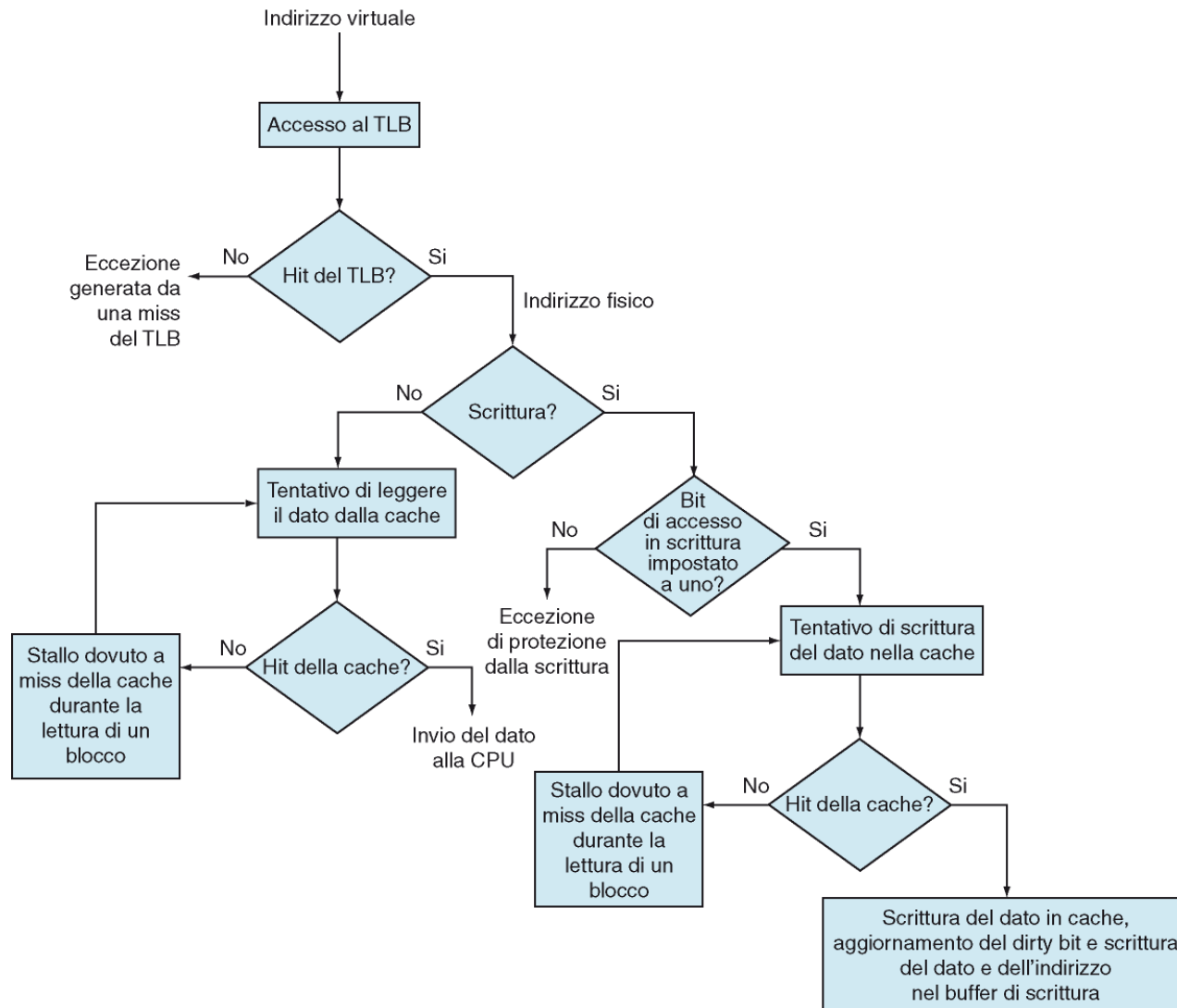
Molto lento

Pro: riduce le traduzioni da indirizzo virtuale a fisico
Contro: Attenzione ai processi multipli

Tra parentesi quadre, i passaggi opzionali

Pro: semplice in caso di multi-tasking fra processi
Contro: obbligo di fare tutte le traduzioni da virtuale a fisico

Fasi dell'accesso (cache ind. fisico)



Casistica (caching indirizzi fisici)

TLB	Tabella delle pagine	Cache	Possibile? Se sì, in quale situazione?
Hit	Hit	Miss	Possibile, sebbene la tabella delle pagine non venga mai realmente controllata se si verifica una hit del TLB.
Miss	Hit	Hit	Miss del TLB, ma l'elemento si trova nella tabella delle pagine; al secondo tentativo, il dato viene trovato nella cache.
Miss	Hit	Miss	Miss del TLB, ma l'elemento si trova nella tabella delle pagine; al secondo tentativo, si verifica però una miss della cache.
Miss	Miss	Miss	Miss del TLB, seguita da un page fault; al secondo tentativo, l'accesso al dato deve provocare una miss della cache.
Hit	Miss	Miss	Impossibile: il TLB non può fornire la traduzione di una pagina che non è presente in memoria.
Hit	Miss	Hit	Impossibile: il TLB non può fornire la traduzione di una pagina che non è presente in memoria.
Miss	Miss	Hit	Impossibile: i dati non possono trovarsi nella cache se la pagina non è presente in memoria.

Casistica (caching indirizzi fisici)

TLB	Tabella delle pagine	Cache	Possibile? Se sì, in quale situazione?
Hit	Hit	Miss	Possibile, sebbene la tabella delle pagine non venga mai realmente controllata se si verifica una hit del TLB.
Miss	Hit	Hit	Miss del TLB, ma l'elemento si trova nella tabella delle pagine; al secondo tentativo, il dato viene trovato nella cache.
Miss	Hit	Miss	Miss del TLB, ma l'elemento si trova nella tabella delle pagine; al secondo tentativo, si verifica però una miss della cache.
Miss	Miss	Miss	Miss del TLB, seguita da un page fault; al secondo tentativo, l'accesso al dato deve provocare una miss della cache.
Hit	Miss	Miss	Impossibile: il TLB non può fornire la traduzione di una pagina che non è presente in memoria.
Hit	Miss	Hit	Impossibile: il TLB non può fornire la traduzione di una pagina che non è presente in memoria.
Miss	Miss	Hit	Impossibile: i dati non possono trovarsi nella cache se la pagina non è presente in memoria.

Supporto del Sistema Operativo

Unico gestore delle **pagine fisiche**

- Risponde alle **eccezioni di Page Fault** caricando in memoria fisica le pagine virtuali
- Assegna le **nuove pagine fisiche** ai processi che le richiedono
- Recupera le **pagine dai processi** che hanno terminato l'esecuzione (o che le rilasciano)

Unico modulo con diritto di modifica sulle **Page Table** dei processi

- Impedisce che il processo possa accedere alle pagine di altri processi
- Svuota il TLB per il **process switch**
 - oppure il TLB contiene l'ID del processo (PID). Concat [PID | TAG]

La CPU deve impedire che il programma possa fare operazioni «pericolose» e gira in 2 modi:

- **«user mode»** per i programmi utente
 - Non sono disponibili le istruzioni per modificare Page Table e Process Table, Interruzioni ...
- **«kernel mode»** per il solo Sistema Operativo (tutte le istruzioni sono disponibili)

Quanto è grande la Page Table?

Esempio: Pagine di 4 Kibyte (2^{10+2} bit), indirizzi di 32 bit: 4 byte per ogni elemento di Page Table

- 12 bit di offset
- 20 bit per il N° di pagina virtuale → 1 «Mebi pagine» (~ 4MiB per processo, $2^{20} \times 4B$)

Come ridurre la Page Table a seconda del comportamento dello spazio di memoria?

- Puntatore all'inizio + lunghezza
 - Se cresce linearmente
- 2 page table con 2 **segmenti** (con puntatori ai 2 estremi e 2 lunghezze)
 - Se è divisibile in due parti che crescono linearmente (ad esempio, stack + heap)
- **Funzione di hashing** che conduce ad uno spazio pari al numero di blocchi fisici
 - Inverted page table
- **Tabella strutturata gerarchicamente** (albero di sotto-tabelle)
 - Pro: OK per spazi di memoria **sparsi**
 - Contro: il **numero di accessi** alla memoria aumenta per ogni livello dell'albero
- **Page table in memoria virtuale** (tabella paginata)
 - Pro: Solo le parti **necessarie** saranno in memoria
 - Contro: **3 accessi** in memoria (page table, N° di pagina, dato)
 - Nota: alcune pagine devono restare sempre in RAM (evita catena infinita di page fault)



Confronto tra livelli diversi

Caratteristica	Valori tipici per una cache L1	Valori tipici per una cache L2	Valori tipici per una memoria a pagine	Valori tipici per un TLB
Dimensione totale in blocchi	250-2000	15 000-50 000	16 000-250 000	40-1024
Dimensione totale in kilobyte	16-64	500-4000	1 000 000-1 000 000 000	0,25-16
Dimensione del blocco in byte	16-64	64-128	4000-64 000	4-32
Penalità di miss in cicli di clock	10-25	100-1000	10 000 000-100 000 000	10-1000
Frequenza di miss (globale per L2)	2%-5%	0,1%-2%	0,00001-0,0001%	0,01%-2%

Esercizio per casa:

- Dato un sistema di memoria a più livelli formato da
 - **CPU** clock a 2 GHz con 3 CPI
 - **Cache L1** Tempo di HIT = 5 ns, percentuale di MISS = 3%
 - **Cache L2** Tempo di HIT = 30ns, percentuale di MISS = 1%
 - **RAM** Tempo di lettura blocco (MISS su L2) = 200 ns
 - **TLB** Tempo di HIT = 1 ns, percentuale di MISS = 0.5%
 - **Memoria Virtuale** Tempo di Page Fault = 3 secondi, Percentuale di MISS = 0,0001%
1. Qual è il tempo totale di accesso per 1 milione (1 000 000) di accessi?
 2. Quante istruzioni vengono svolte in media per ogni accesso?

NOTA: ignorare i tempi di scrittura delle pagine su disco necessari per il Write Back

